APEXPRICING

Retailer's User Guide — Turning AI Pricing into Store Profit

AI Dynamic Pricing & Inventory Lifecycle Optimization · Indian Consumer Electronics Retail · Demo Edition

1. What This Tool Does For You

Every phone sitting on your shelf costs you money twice: working-capital interest and storage, plus the price collapse when the next model launches. This engine tells you, for every SKU in every store, the selling price that maximizes total profit across the product's remaining life — not just today's margin.

2. The Core Idea: Lifecycle Profit, Not Today's Margin

Traditional pricing asks "what margin do I make on this unit?" ApexPricing asks "how much total profit will this stock generate before the successor arrives?" It adds up:

- + gross margin on expected sales,
- − holding and financing cost of stock sitting in stores,
- − depreciation write-down on units left stranded after the successor launches.

The flagship demo SKU shows why this matters:

Price	Daily Demand	Stock Cleared (60 days)	Leftover Units	Expected Profit
₹79,999 (status quo)	10.2 u/day	611 units	188 (≈₹38L write-down)	₹18.2 Lakh
₹79,000 (AI recommended)	12.4 u/day	745 units	54 units	₹56.5 Lakh

A tiny ₹999 price cut triples lifecycle profit because it avoids most of the end-of-life loss. That is the engine's whole philosophy: small, early, data-driven moves beat big, late, panic markdowns.

3. Your Playbook, Screen by Screen

Screen	It answers	Your action
Portfolio Summary	Which SKUs are burning money right now?	Work the top-3 opportunities table first; check the "Pre-Successor At Risk" KPI.
SKU Matrix	What price should each SKU be at?	Filter by Price Increases to capture margin; filter by Clearance Markdown / High Risk to act today.
Elasticity Simulator	What if I match Flipkart, or hold?	Drag the price slider and watch Horizon Net Profit and clearance time before committing.

Market Intelligence	Am I premium or underpriced vs Amazon/Flipkart/Croma?	Defend justified premium pricing; fix accidental under-pricing.
Store Balancing	Which store is stockout-bound while another sits on stock?	Approve profitable transfers (₹250/unit logistics vs margin saved).
Explain (per SKU)	Why does the model say this?	Read the contribution bars: competitor stockout, inventory pressure, successor proximity.
Simulate Launch Signal	A successor just got announced — what now?	Trigger the event and execute the new markdown immediately, before market prices collapse.

4. A Real Walkthrough: iPhone 17 256GB

- **1. Open Portfolio Summary:** 800 units in stock, cost ₹68,000, selling at ₹79,999, successor in 60 days. The engine detects the clearance shortfall (611 sold vs 800 in stock) and the ₹38 Lakh depreciation risk.
- **2. Check the Elasticity Simulator:** The demand curve shows ₹78,999–79,000 as the sweet spot — the model predicts 12.4 units/day there.
- **3. Click Explain:** The SHAP bars show competitor pricing pressure is mild and inventory pressure dominates — a small cut, not a fire-sale, is optimal.
- **4. Execute the recommendation:** ₹79,000 — a ₹999 reduction, 93% clearance, +₹38.3 Lakh vs status quo. Matching Flipkart at ₹76,999 would LOSE money; a 15% blanket markdown would go below cost (forbidden).
- **5. Track and iterate:** Monitor daily velocity for a week; if clearance lags, the simulator recalculates the next step.

5. Your Weekly Routine

When	Do
Daily	SKU Matrix → High-Risk rows; Sync Data; execute clearances and price increases.
Monday	Portfolio Summary → top-3 opportunities; Store Balancing → approve transfer plan.
After any competitor drop	Elasticity Simulator → test matching vs holding before reacting.
After any launch announcement	Simulate Launch Signal → execute the re-optimized markdown the same day.
Month end	Recalculate Engine → compare predicted vs actual; feed deviations back into the data.

6. Going Live With Your Own Data

The demo runs on simulated data. To make real decisions, feed the engine your POS/ERP exports — six CSV files with these columns:

File	What it holds
products.csv	Your catalog: SKU id, brand/model, MRP, successor model, launch & successor dates.
stores.csv	Your branches: id, name, city, footfall index.
costs.csv	Purchase cost, landed cost, holding/financing/depreciation rates per SKU.
competitors.csv	Daily competitor prices & stock availability (Amazon, Flipkart, Croma...).
sales.csv	Daily transactions: store, SKU, selling price, units, revenue.
inventory.csv	Daily stock per store-SKU: opening, sold, received, closing.

Steps: (1) fill the templates in data/import_templates, (2) copy them over data/synthetic or repoint src/config.py, (3) run python main.py --train, (4) open the dashboard and click Recalculate Engine.

Tip: the model learns price sensitivity from price VARIATION — include your past markdowns, promos and price increases. A flat price history makes elasticity honestly "Not Identifiable".

7. Rules the Engine Never Breaks

- **Legal ceiling:** Selling price ≤ MRP at all times (Legal Metrology Packaged Commodities Rules).
- **Margin floor:** No recommendation below acquisition cost — loss tolerance is 0 by default.
- **Honesty:** If data cannot identify price sensitivity, the engine says so instead of inventing a number.
- **Explainability:** Every recommendation comes with model-level explanation (SHAP contribution bars).

8. What It Cannot Do Yet

- Live competitor scraping (currently you update competitors.csv manually).
- Your exact store/SKU specifics until real data is imported.
- Replacement for your judgment on promotions, festive pricing, and customer relationships — it is a decision aid, not autopilot.